

INTERVIEW PREPARATION

DSA Concepts & Practice

AI Engineer ke liye — Roman Urdu mein Asaan
Explanation

Array & String — Practical Coding Questions (Python)

HashMap • Stack • Queue • Linked List • Tree • Graph

BFS • DFS • Heap • Binary Search • Topological Sort • Basic DP

Goal: Interview ke liye concept clear karna, confidence build karna

Intro — Ye Guide Kaise Use Karein

Assalam-o-Alaikum! Aap AI Engineer hain aur abhi tak sirf 6 mahine ka internship experience hai, aur ab job hunt kar rahe hain. Bohat companies, chahe AI/ML role hi kyun na ho, basic DSA (Data Structures & Algorithms) ka round zaroor lete hain — kyunke wo dekhna chahte hain ke aapki problem-solving aur coding fundamentals kitni strong hain.

Ye guide do hisso mein divide hai:

- **Array aur String** — In dono par sab se zyada practical coding questions puchay jate hain, is liye in ke liye **hands-on Python code** ke sath solved questions diye gaye hain (jaise: 2nd largest element, 3rd largest element, max/min element, waghera).
- **Baaki 11 topics** (HashMap, Stack, Queue, Linked List, Tree, Graph, BFS, DFS, Heap, Binary Search, Topological Sort, Basic DP) — In ka **concept theory** Roman Urdu mein clearly samjhaya gaya hai, taake interview mein jab wo pochein "iska use kahan hota hai" ya "ye kaise kaam karta hai", to aap confidently jawab de sakein.

Tip: Sirf padhna kaafi nahi hai. Har Array/String question ko khud se, bina dekhe, ek baar zaroor likh kar practice karein. Isse muscle memory ban jati hai aur interview mein pressure kam hota hai.

Interview Strategy: Jab bhi koi coding question mile, seedha code likhna start na karein. Pehle: (1) Question ko apne alfaaz mein repeat karein, (2) Edge cases pochein (empty array? negative numbers? duplicates?), (3) Approach verbally explain karein, (4) Phir code likhein, (5) Dry-run kar ke test karein. Ye process interviewer ko dikhata hai ke aap sirf code nahi, *soch* rahe hain.

Contents

- 1 Array — Concept + Practical Questions
- 2 String — Concept + Practical Questions
- 3 HashMap — Theory
- 4 Stack — Theory
- 5 Queue — Theory
- 6 Linked List — Theory
- 7 Tree — Theory
- 8 Graph — Theory
- 9 BFS (Breadth First Search) — Theory
- 10 DFS (Depth First Search) — Theory
- 11 Heap — Theory
- 12 Binary Search — Theory
- 13 Topological Sort — Theory
- 14 Basic Dynamic Programming — Theory
- 15 Bonus: Interview Tips & Time Complexity Cheat Sheet

1. Array

PRACTICAL + THEORY

Concept (Roman Urdu)

Array ek **contiguous memory location** hoti hai jisme same data type ke multiple elements store hotay hain, aur har element ko ek **index** (0 se start hone wala number) se access kiya jata hai. Matlab agar aapke paas 5 numbers hain, to wo memory mein ek ke baad ek (side by side) store hotay hain, aur aap seedha index se kisi bhi element tak "O(1)" time mein pohanch saktay hain.

Python mein hum normally `list` use karte hain jo array jaisa hi kaam karti hai (lekin dynamic hai, size fix nahi hoti).

Yaad rakhne wali baatein

- Indexing: `arr[0]` se start hoti hai, `arr[-1]` last element deta hai (Python).
- Access time: O(1) — direct index se access.
- Insert/Delete beech mein: O(n) — kyunke baaki elements shift karne partay hain.
- Search (agar sorted nahi): O(n) — Linear Search. Agar sorted hai to O(log n) — Binary Search.

Common Array Time Complexities: Access O(1) | Search O(n) | Insert/Delete O(n) | Append(end) O(1)

Practical Coding Questions

Q1. Array mein Maximum Element Dhoondein

Approach (Roman Urdu): Ek variable `max_val` banayein aur usko array ke pehle element se initialize karein. Phir puray array ko loop karein — jahan bhi koi element `max_val` se bara mile, usko update kar dein.

```
def find_max(arr):
    if len(arr) == 0:
        return None
    max_val = arr[0]
    for num in arr[1:]:
        if num > max_val:
            max_val = num
    return max_val

# Test
arr = [3, 7, 2, 9, 4, 1]
print(find_max(arr)) # Output: 9

# Python built-in shortcut: max(arr)
```

Time: O(n) | Space: O(1)

Q2. Array mein Minimum Element Dhoondein

Bilkul same logic jaisa maximum ka tha, bas comparison sign ulta ho jayega.

```
def find_min(arr):
    if len(arr) == 0:
        return None
    min_val = arr[0]
```

```

    for num in arr[1:]:
        if num < min_val:
            min_val = num
    return min_val

arr = [3, 7, 2, 9, 4, 1]
print(find_min(arr)) # Output: 1

# Python built-in shortcut: min(arr)

```

Time: $O(n)$ | Space: $O(1)$

Q3. Array mein 2nd Largest Element Dhoondein

Approach: Do variables rakhein: `largest` aur `second_largest`, dono ko negative infinity se initialize karein. Array ko ek baar loop karein:

- Agar current number `largest` se bara hai → `second_largest = largest`, phir `largest = current`.
- Warna agar current number `second_largest` se bara hai (lekin largest ke barabar nahi) → `second_largest = current`.

Important: Duplicates ka khayal rakhein — agar array mein sab elements same hain (jaise [5,5,5]), to 2nd largest exist nahi karta.

```

def second_largest(arr):
    if len(arr) < 2:
        return None

    largest = float('-inf')
    second = float('-inf')

    for num in arr:
        if num > largest:
            second = largest
            largest = num
        elif num > second and num != largest:
            second = num

    return second if second != float('-inf') else None

arr = [12, 35, 1, 10, 34, 1]
print(second_largest(arr)) # Output: 34

```

Time: $O(n)$ | Space: $O(1)$ — sirf ek pass mein solve ho jata hai, sorting ki zaroorat nahi (sorting se $O(n \log n)$ lagta)

Interview mein bolne wali baat: "Main array ko sort bhi kar sakta tha ($O(n \log n)$) aur dusra last element utha leta, lekin single-pass approach $O(n)$ mein zyada efficient hai."

Q4. Array mein 3rd Largest Element Dhoondein

Approach: Same jaisa 2nd largest tha, bas ab 3 variables rakhne parenge: `first`, `second`, `third`. Jab bhi naya number in teeno se bara ho, usko sahi jagah insert karein aur baqi ko shift kar dein.

```

def third_largest(arr):
    unique = list(set(arr)) # duplicates hata dein
    if len(unique) < 3:
        return None

    first = second = third = float('-inf')

```

```

for num in unique:
    if num > first:
        third = second
        second = first
        first = num
    elif num > second:
        third = second
        second = num
    elif num > third:
        third = num

return third

arr = [12, 35, 1, 10, 34, 1]
print(third_largest(arr)) # Output: 12

```

Time: $O(n)$ | Space: $O(n)$ (duplicates remove karne ke liye set banaya)

General Pattern: "Kth largest element" ka ek generic tareeqa hai — `sorted(arr, reverse=True)[k-1]`, ya phir **Min-Heap** of size k use karna (large datasets ke liye better, $O(n \log k)$). Heap wala tareeqa Heap section mein explain hai.

Q5. Array Reverse Karna

Approach (Two Pointer Technique): Ek pointer `left` start par aur ek `right` end par rakhein. Dono ke elements swap karein aur pointers ko andar ki taraf move karein, jab tak wo cross na ho jayein.

```

def reverse_array(arr):
    left, right = 0, len(arr) - 1
    while left < right:
        arr[left], arr[right] = arr[right], arr[left]
        left += 1
        right -= 1
    return arr

arr = [1, 2, 3, 4, 5]
print(reverse_array(arr)) # Output: [5, 4, 3, 2, 1]

# Python shortcut: arr[::-1]

```

Time: $O(n)$ | Space: $O(1)$ — in-place reverse (Two Pointer technique interviews mein bohat pasand ki jati hai)

Q6. Array mein Duplicate Element Dhoondein

Approach (HashSet use kar ke): Ek empty set banayein. Array ko loop karein — agar element pehle se set mein maujood hai, matlab wo duplicate hai. Warna set mein add kar dein.

```

def find_duplicates(arr):
    seen = set()
    duplicates = []
    for num in arr:
        if num in seen:
            duplicates.append(num)
        else:
            seen.add(num)
    return duplicates

```

```
arr = [4, 3, 2, 7, 8, 2, 3, 1]
print(find_duplicates(arr)) # Output: [2, 3]
```

Time: $O(n)$ | Space: $O(n)$ — HashSet ke wajah se lookup $O(1)$ mein hota hai, is liye ye Brute Force ($O(n^2)$, do loops) se kaafi behtar hai

Q7. Two Sum Problem (Bohat Common Interview Question)

Question: Ek array aur ek `target` number diya hai. Do aise elements dhoondein jinka sum target ke barabar ho, aur unke indices return karein.

Approach: HashMap use karein. Har number ke liye check karein ke `target - number` HashMap mein pehle se maujood hai ya nahi. Agar hai, to jawab mil gaya. Warna current number ko HashMap mein store kar dein.

```
def two_sum(arr, target):
    seen = {} # value -> index
    for i, num in enumerate(arr):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []

arr = [2, 7, 11, 15]
print(two_sum(arr, 9)) # Output: [0, 1] (2 + 7 = 9)
```

Time: $O(n)$ | Space: $O(n)$ — Brute force do nested loops se $O(n^2)$ lagta, HashMap se sirf ek pass mein $O(n)$ mein solve hota hai

Ye question kyun important hai: Ye almost har company puchti hai kyunke isse pata chalta hai ke candidate ko pata hai brute force se optimal solution tak kaise pohanchna hai (HashMap trade-off: extra space use kar ke time bachana).

Q8. Zeros ko Array ke End mein Move Karna (Order Maintain karte hue)

```
def move_zeros(arr):
    insert_pos = 0
    for i in range(len(arr)):
        if arr[i] != 0:
            arr[insert_pos], arr[i] = arr[i], arr[insert_pos]
            insert_pos += 1
    return arr

arr = [0, 1, 0, 3, 12]
print(move_zeros(arr)) # Output: [1, 3, 12, 0, 0]
```

Time: $O(n)$ | Space: $O(1)$ — Two Pointer technique (ek pointer non-zero elements place karne ke liye)

Q9. Maximum Subarray Sum (Kadane's Algorithm)

Question: Ek array diya hai (jisme negative numbers bhi ho saktay hain). Contiguous subarray ka maximum sum dhoondein.

Approach: Do variables rakhein — `current_sum` aur `max_sum`. Har element par decide karein: ya to current element ko purani subarray sum mein add karein, ya phir current element se hi nayi subarray start karein (jab purani sum negative ho jaye, usko carry karna nuksaan deh hai).

```
def max_subarray_sum(arr):
    current_sum = max_sum = arr[0]
    for num in arr[1:]:
        current_sum = max(num, current_sum + num)
        max_sum = max(max_sum, current_sum)
    return max_sum

arr = [-2, 1, -3, 4, -1, 2, 1, -5, 4]
print(max_subarray_sum(arr)) # Output: 6 (subarray [4, -1, 2, 1])
```

Time: $O(n)$ | Space: $O(1)$ — ye Dynamic Programming ka classic example hai (Basic DP section mein iska concept bhi milega)

Q10. Array ko K Positions se Rotate Karna

```
def rotate_array(arr, k):
    n = len(arr)
    k = k % n # agar k array ke size se bara ho
    return arr[-k:] + arr[:-k]

arr = [1, 2, 3, 4, 5, 6, 7]
print(rotate_array(arr, 3)) # Output: [5, 6, 7, 1, 2, 3, 4]
```

Time: $O(n)$ | Space: $O(n)$ — In-place version bhi ban sakta hai teen reversals se ($O(1)$ space), jo interview mein bonus point hai

Extra Practice Questions (khud try karein): Missing number in array (1 to n), Union & Intersection of two arrays, Move all negative numbers to one side, Find pairs with given sum, Trapping Rain Water problem, Best Time to Buy/Sell Stock.

2. String

PRACTICAL + THEORY

Concept (Roman Urdu)

String basically characters ka ek sequence hoti hai (jaise array of characters). Python mein strings **immutable** hoti hain — matlab ek baar string ban jaye to uske individual characters ko directly change nahi kar saktay (`s[0] = 'a'` error dega). Agar modify karna ho to naya string banana parta hai, ya `list(s)` mein convert kar ke phir wapas `''.join()` karna parta hai.

Yaad rakhne wali baatein

- Strings immutable hain (Python mein) — modification ke liye naya string banta hai.
- Slicing bohat useful hai: `s[::-1]` = reverse, `s[1:4]` = substring.
- ASCII/character frequency count karne ke liye HashMap (dictionary) ya `collections.Counter` use hota hai.

Practical Coding Questions

Q1. String Reverse Karna

```
def reverse_string(s):
    left, right = 0, len(s) - 1
    s = list(s) # string ko list mein convert (kyunke immutable hai)
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
    return ''.join(s)

print(reverse_string("hello")) # Output: "olleh"

# Python shortcut: s[::-1]
```

Time: O(n) | Space: O(n)

Q2. Palindrome Check (kya string dono taraf se same parhi jati hai)

```
def is_palindrome(s):
    s = s.lower().replace(" ", "") # case aur spaces ignore karein
    left, right = 0, len(s) - 1
    while left < right:
        if s[left] != s[right]:
            return False
        left += 1
        right -= 1
    return True

print(is_palindrome("Madam")) # True
print(is_palindrome("A man a plan a canal Panama")) # True
print(is_palindrome("hello")) # False
```

Time: O(n) | Space: O(1) (agar in-place check karein) — Two Pointer technique

Q3. Anagram Check (kya do strings ek dusre ke letters se bani hain)

Approach: Agar dono strings sort karne ke baad barabar hain, to wo anagram hain. Ya phir character frequency count kar ke compare karein (zyada efficient).

```
def is_anagram(s1, s2):
    if len(s1) != len(s2):
        return False
    return sorted(s1) == sorted(s2)

# Better approach using frequency count (HashMap):
from collections import Counter
def is_anagram_v2(s1, s2):
    return Counter(s1) == Counter(s2)

print(is_anagram("listen", "silent")) # True
print(is_anagram("hello", "world"))  # False
```

Sorting approach: $O(n \log n)$ | Frequency count approach: $O(n)$ — HashMap wala tareeqa interview mein zyada impress karta hai

Q4. Pehla Non-Repeating Character Dhoondein

```
from collections import Counter

def first_non_repeating(s):
    freq = Counter(s)
    for ch in s:
        if freq[ch] == 1:
            return ch
    return None

print(first_non_repeating("swiss")) # Output: "w"
```

Time: $O(n)$ | Space: $O(n)$ — pehle HashMap se frequency count, phir dobara loop kar ke pehla unique character dhoonda

Q5. Vowels aur Consonants Count Karna

```
def count_vowels_consonants(s):
    vowels = set("aeiouAEIOU")
    vowel_count = 0
    consonant_count = 0
    for ch in s:
        if ch.isalpha():
            if ch in vowels:
                vowel_count += 1
            else:
                consonant_count += 1
    return vowel_count, consonant_count

v, c = count_vowels_consonants("Hello World")
print(f"Vowels: {v}, Consonants: {c}") # Vowels: 3, Consonants: 7
```

Time: $O(n)$ | Space: $O(1)$

Q6. String Compression (jaise "aaabbc" → "a3b2c1")

```
def compress_string(s):
    if not s:
        return s
    result = []
    count = 1
    for i in range(1, len(s)):
        if s[i] == s[i - 1]:
            count += 1
        else:
            result.append(s[i - 1] + str(count))
            count = 1
    result.append(s[-1] + str(count))
    compressed = ''.join(result)
    # agar compressed version original se bara ho, to original return karein
    return compressed if len(compressed) < len(s) else s

print(compress_string("aaabbc")) # Output: "a3b2c1"
```

Time: $O(n)$ | Space: $O(n)$

Q7. Do Strings ka Longest Common Prefix

```
def longest_common_prefix(strs):
    if not strs:
        return ""
    prefix = strs[0]
    for s in strs[1:]:
        while not s.startswith(prefix):
            prefix = prefix[:-1]
        if not prefix:
            return ""
    return prefix

print(longest_common_prefix(["flower", "flow", "flight"])) # "fl"
```

Time: $O(n*m)$ jahan n = strings ki tadad, m = average length

Extra Practice Questions: String mein har character ki frequency count karna, Longest substring without repeating characters, Valid parentheses (Stack use hota hai), String ko words mein split kar ke reverse karna, Caesar cipher / simple encryption.

3. HashMap (Dictionary)

THEORY

HashMap ek aisi data structure hai jo data ko **key-value pairs** ki form mein store karti hai. Python mein isko `dict` kehte hain. Sab se important baat ye hai ke HashMap mein **average case mein $O(1)$ time** mein insert, delete, aur search ho jata hai — chahe data kitna bhi bara kyun na ho.

Ye Kaam Kaise Karta Hai?

HashMap internally ek **Hash Function** use karta hai jo key ko ek number (hash) mein convert karti hai, aur wo number decide karta hai ke data memory mein kahan store hoga (kis "bucket" mein). Is liye jab aap koi key search karte hain, to system seedha us bucket tak pohanch jata hai, poore data ko loop karne ki zaroorat nahi parti — yehi wajah hai ke ye itna fast hota hai.

Collision: Kabhi kabhi do different keys ka hash same nikal ata hai — isko *collision* kehte hain. Isko handle karne ke liye "chaining" (same bucket mein linked list) ya "open addressing" techniques use hoti hain. Interview mein agar puchein "hash collision kaise handle hoti hai" to yehi jawab dena hai.

Kab Use Karein?

- Jab aapko fast lookup chahiye ho (jaise: "kya ye element pehle se maujood hai?").
- Frequency counting (kisi cheez ki tadad count karna).
- Do datasets ke beech relationship store karna (jaise: Two Sum problem mein value $\rightarrow$ index).
- Caching / Memoization (Dynamic Programming mein bohat use hota hai).

Average Case: Insert $O(1)$ | Delete $O(1)$ | Search $O(1)$ | Worst Case (bohat collisions ho jayein): $O(n)$

Interview Example: "Mujhe array mein duplicates dhoondne hain" — Brute force $O(n^2)$ hoga (har element ko baaki sab se compare karna), lekin HashMap/HashSet use kar ke $O(n)$ mein solve ho jata hai, kyunke lookup $O(1)$ hai.

4. Stack

THEORY

Stack ek **LIFO (Last In, First Out)** data structure hai. Matlab jo element sab se aakhir mein add hua ho, wahi sab se pehle nikalta hai. Isko aap plates ke stack ki tarah imagine kar saktay hain — jo plate sab se upar (last) rakhi thi, wahi sab se pehle uthai jayegi.

Main Operations

- **push(x)** — Stack ke top par ek naya element add karna.
- **pop()** — Stack ke top se element remove karna aur return karna.
- **peek() / top()** — Top ka element dekhna, bina remove kiye.
- **isEmpty()** — Check karna ke stack khali hai ya nahi.

Python mein Stack ko simple list se implement kar saktay hain: `append()` = push, `pop()` = pop.

```
stack = []
stack.append(10)    # push
stack.append(20)
stack.append(30)
print(stack.pop())  # 30 (last in, first out)
print(stack[-1])    # peek -> 20
```

Push: O(1) | Pop: O(1) | Peek: O(1)

Real-World Use Cases (Interview mein ye examples bohat useful hain)

- **Function Call Stack:** Jab bhi koi function call hota hai, wo stack mein push hota hai; jab return hota hai, pop ho jata hai. Recursion isi principle par kaam karta hai.
- **Undo/Redo** feature (text editors mein).
- **Browser History** (Back button).
- **Balanced Parentheses Check** — jaise `"(a+b)*(c-d)"` mein brackets match kar rahe hain ya nahi, ye check karna Stack ka classic use case hai.
- **Expression Evaluation** (jaise Infix to Postfix conversion).

Common Interview Question: "Valid Parentheses" — Har opening bracket ko stack mein push karo. Jab closing bracket mile, to stack ke top se pop kar ke check karo ke wo match karta hai ya nahi. Agar end mein stack empty hai, to string valid hai.

5. Queue

THEORY

Queue ek **FIFO (First In, First Out)** data structure hai. Matlab jo element sab se pehle add hua ho, wahi sab se pehle nikalta hai — bilkul ek line/qatar ki tarah, jaise bank mein log line lagate hain, jo pehle aya wahi pehle serve hota hai.

Main Operations

- **enqueue(x)** — Queue ke end (rear) mein element add karna.
- **dequeue()** — Queue ke front se element remove karna.
- **front()** — Front ka element dekhna, bina remove kiye.

Python mein Queue ke liye `collections.deque` use karna best hai (list ke `pop(0)` se $O(n)$ lagta hai, lekin deque se $O(1)$):

```
from collections import deque

queue = deque()
queue.append(10)    # enqueue
queue.append(20)
queue.append(30)
print(queue.popleft()) # dequeue -> 10 (first in, first out)
```

Enqueue: $O(1)$ | Dequeue: $O(1)$ — (deque use karte hue; simple list se dequeue $O(n)$ hoga)

Real-World Use Cases

- **BFS (Breadth First Search)** mein Queue ka use hota hai — ye sab se important use case hai.
- Printer job scheduling, CPU task scheduling.
- Customer service / ticket systems (jo pehle aya usko pehle serve karna).

Variations jo interview mein puchi ja sakti hain: **Circular Queue** (jab end tak pohanchne ke baad wapas start se use ho), **Priority Queue** (Heap se implement hoti hai — jahan sab se important/urgent item pehle nikalta hai, order ke bajaye priority ke hisab se), **Deque** (dono taraf se add/remove ho sakta hai).

6. Linked List

THEORY

Linked List bhi ek linear data structure hai (array ki tarah), lekin farq ye hai ke iske elements memory mein **contiguous (side by side)** nahi hote. Har element ek **Node** hota hai jisme do cheezein hoti hain: (1) **data**, (2) **next** — jo agle node ka address/pointer store karta hai.

Node Structure (Python)

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
```

| Array vs Linked List — Ye Farq Interview Mein Zaroor Puchte Hain

- **Array:** Fixed/contiguous memory, index se $O(1)$ access, lekin beech mein insert/delete $O(n)$ (shifting ki wajah se).
- **Linked List:** Memory kahin bhi ho sakti hai (non-contiguous), index se direct access nahi ($O(n)$, traverse karna parta hai), lekin agar aapke paas node ka reference hai to insert/delete $O(1)$ mein ho sakta hai (kyunke sirf pointers change hotay hain, shifting nahi).

| Types

- **Singly Linked List** — Har node sirf agle node ki taraf point karta hai.
- **Doubly Linked List** — Har node ke paas `next` aur `prev` dono hotay hain, is liye dono directions mein traverse kar saktay hain.
- **Circular Linked List** — Last node wapas first node ki taraf point karta hai (loop bann jata hai).

Access: $O(n)$ | Search: $O(n)$ | Insert at head: $O(1)$ | Insert at end: $O(n)$ (agar tail pointer na ho) | Delete: $O(1)$ agar node reference maujood ho

Common Interview Questions: Linked List ko reverse karna, Cycle detect karna (Floyd's Cycle Detection — "slow and fast pointer" technique — do pointers use karo, ek 1 step aur ek 2 step move kare, agar kabhi dono mil jayein to cycle hai), Middle node dhoondna (same slow-fast pointer technique se), Do sorted linked lists merge karna.

7. Tree

THEORY

Tree ek **hierarchical (non-linear)** data structure hai jisme ek "root" node se start hoti hai aur us se "children" nodes branch out hotay hain. Family tree ki tarah imagine karein — top par ek ancestor, phir uske bachay, phir unke bachay, waghera.

Basic Terminology

- **Root** — Sab se upar wala node (jahan se tree start hoti hai).
- **Parent / Child** — Jo node upar hai wo parent, niche wala child.
- **Leaf** — Wo node jiska koi child na ho (sab se aakhir mein).
- **Height** — Root se sab se door wale leaf tak ka distance.

Binary Tree vs Binary Search Tree (BST)

Binary Tree: Har node ke max 2 children ho saktay hain (left aur right), koi order rule nahi.

Binary Search Tree (BST): Binary Tree hi hai, lekin ek special property ke sath: har node ke **left** subtree ki saari values us node se **choti** hoti hain, aur **right** subtree ki saari values us node se **bari** hoti hain. Isi property ki wajah se BST mein search bohat fast hoti hai.

Node Structure (Python)

```
class TreeNode:
    def __init__(self, val):
        self.val = val
        self.left = None
        self.right = None
```

Tree Traversal (Bohat Important — Interview Favorite)

- **Inorder (Left → Root → Right):** BST mein ye sorted order deta hai.
- **Preorder (Root → Left → Right):** Tree copy karne ya structure serialize karne ke liye useful.
- **Postorder (Left → Right → Root):** Tree delete karne ke liye useful (pehle children delete, phir parent).
- **Level Order:** Tree ko level by level traverse karna — ye BFS use karta hai (Queue ke sath).

Balanced BST mein: Search/Insert/Delete = $O(\log n)$ | Unbalanced/Worst case (linked-list jaisi bann jaye): $O(n)$

AVL Tree / Red-Black Tree: Ye "self-balancing" BSTs hain jo automatically balance maintain karte hain taake $O(\log n)$ guarantee rahe. Naam sunna kaafi hai interview ke liye, deep implementation zaroori nahi entry-level role ke liye.

8. Graph

THEORY

Graph ek data structure hai jisme **nodes (vertices)** hote hain jo **edges** se connect hotay hain. Tree bhi actually ek special type ka graph hai (jisme cycle nahi hoti). Graph zyada flexible hai — koi bhi node kisi bhi node se connect ho sakta hai, aur cycles bhi ho sakti hain.

| Types of Graphs

- **Directed vs Undirected:** Directed mein edges ki ek direction hoti hai ($A \rightarrow B$, matlab A se B ja saktay hain, zaroori nahi B se A). Undirected mein edge dono directions mein kaam karta hai.
- **Weighted vs Unweighted:** Weighted graph mein har edge ka ek "cost/weight" hota hai (jaise: do cities ke beech distance). Unweighted mein sirf connection hoti hai, koi cost nahi.
- **Cyclic vs Acyclic:** Cyclic mein loop ban sakta hai ($A \rightarrow B \rightarrow C \rightarrow A$), Acyclic mein nahi (DAG = Directed Acyclic Graph, jo Topological Sort mein use hota hai).

| Graph Represent Kaise Karte Hain

- **Adjacency List** — Har node ke liye uske connected nodes ki list (dictionary/hashmap use hoti hai). Space-efficient, sparse graphs ke liye best. Ye sab se common approach hai coding interviews mein.
- **Adjacency Matrix** — 2D matrix jahan `matrix[i][j] = 1` agar node i aur j connected hain. Dense graphs ke liye theek, lekin space $O(V^2)$ leta hai.

```
# Adjacency List example (Python dictionary se)
graph = {
    'A': ['B', 'C'],
    'B': ['A', 'D'],
    'C': ['A', 'D'],
    'D': ['B', 'C']
}
```

| Real-World Use Cases

- Social networks (log = nodes, friendship = edges).
- Google Maps / shortest path (Dijkstra's Algorithm).
- Recommendation systems, Web page linking (Google's PageRank).
- Network routing, dependency resolution (jaise package managers).

Graph ko traverse karne ke do main tareeqe hain: **BFS** aur **DFS** — jo agle do sections mein detail se explain hain.

9. BFS (Breadth First Search)

THEORY

BFS ek traversal algorithm hai jo Tree ya Graph ko **level by level** explore karta hai. Matlab pehle current node ke saare "immediate neighbors" visit karta hai, phir unke neighbors, aur is tarah aage barhta hai — jaise pani mein pathar phenkne se banne wali circles, jo center se bahar ki taraf phailti hain.

Kaise Kaam Karta Hai

BFS **Queue** data structure use karta hai (FIFO). Algorithm ye hai:

1. Starting node ko queue mein push karein aur "visited" mark karein.
2. Jab tak queue khali na ho: queue se ek node nikalein (dequeue), uske saare unvisited neighbors ko queue mein add karein aur visited mark karein.

```
from collections import deque

def bfs(graph, start):
    visited = set([start])
    queue = deque([start])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            if neighbor not in visited:
                visited.add(neighbor)
                queue.append(neighbor)
    return result

graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}
print(bfs(graph, 'A')) # Output: ['A', 'B', 'C', 'D']
```

Time: $O(V + E)$ — V = vertices, E = edges | Space: $O(V)$ — queue aur visited set ke liye

Kab Use Karein

- **Shortest Path** dhoondne ke liye (unweighted graph mein) — kyunke BFS level-by-level jata hai, jo pehle target milega wahi shortest path hoga.
- Social network mein "connections at distance k" dhoondna (jaise LinkedIn ka "2nd degree connections").
- Web crawling.

Interview mein yaad rakhein: "BFS = Queue + Shortest Path (unweighted)". Ye ek-line summary bohat kaam ati hai jaldi jawab dene ke liye.

10. DFS (Depth First Search)

THEORY

DFS bhi ek traversal algorithm hai, lekin BFS ke bilkul ulta approach leta hai — ye ek path mein **jitna deep ja sakta hai jata hai**, phir jab aage koi rasta na bache to peeche (backtrack) hokar dusra path try karta hai.

Kaise Kaam Karta Hai

DFS **Stack** use karta hai — ya to explicitly (stack data structure se), ya implicitly **recursion** ke through (kyunke recursion khud function call stack use karta hai).

```
# Recursive DFS
def dfs(graph, node, visited=None):
    if visited is None:
        visited = set()
    visited.add(node)
    print(node, end=' ')
    for neighbor in graph[node]:
        if neighbor not in visited:
            dfs(graph, neighbor, visited)
    return visited

graph = {'A': ['B', 'C'], 'B': ['A', 'D'], 'C': ['A', 'D'], 'D': ['B', 'C']}
dfs(graph, 'A') # Output: A B D C
```

Time: $O(V + E)$ | Space: $O(V)$ — recursion stack ke liye (worst case mein)

BFS vs DFS — Interview ka Favorite Comparison Question

BFS Level-by-level, Queue use karta hai, Shortest path (unweighted) ke liye best, zyada memory le sakta hai wide graphs mein.

DFS Deep-first, Stack/Recursion use karta hai, Path existence check karne, cycle detect karne, aur backtracking problems ke liye best, kam memory le sakta hai deep graphs mein.

Kab Use Karein

- Maze solving, puzzle solving (Sudoku, N-Queens — backtracking problems).
- Cycle detection graph mein.
- Topological Sort (agla section) DFS se hi banta hai.
- Connected components dhoondna.

Interview mein yaad rakhein: "DFS = Stack/Recursion + Backtracking/Path problems".

11. Heap

THEORY

Heap ek special **Binary Tree** based data structure hai jo hamesha ek property maintain karti hai: **parent node ka value hamesha uske children se related hota hai** (ya to sab se chota ya sab se bara).

Types

- **Min-Heap:** Root (top) node hamesha sab se **chota** element hota hai. Har parent apne children se chota hota hai.
- **Max-Heap:** Root node hamesha sab se **bara** element hota hai. Har parent apne children se bara hota hai.

Python mein `heapq` module built-in Min-Heap provide karta hai:

```
import heapq

nums = [5, 2, 8, 1, 9]
heapq.heapify(nums)          # O(n) - list ko heap mein convert karta hai
heapq.heappush(nums, 3)      # O(log n) - insert
smallest = heapq.heappop(nums) # O(log n) - sab se chota element nikalna
print(smallest)              # 1

# Max-Heap trick: numbers ko negative kar ke Min-Heap use karein
max_heap = [-x for x in nums]
heapq.heapify(max_heap)
largest = -heapq.heappop(max_heap)
```

Insert: $O(\log n)$ | Get Min/Max (peek): $O(1)$ | Extract Min/Max: $O(\log n)$ | Build heap from array: $O(n)$

Kab Use Karein (Ye Bohat Interview-Relevant Hai)

- **Priority Queue** implement karne ke liye (jahan sab se urgent/important task pehle process ho).
- **"Kth Largest/Smallest Element"** type ke questions — Size-k ka Min-Heap maintain kar ke $O(n \log k)$ mein solve hota hai, jo poora sort karne ($O(n \log n)$) se behtar hai.
- **Median of a stream** of numbers (do heaps ka combination use hota hai).
- **Dijkstra's Shortest Path Algorithm** mein heap use hota hai.
- Task scheduling systems, jaise OS process scheduling.

Interview mein yaad rakhein: "Jab bhi question mein 'top k', 'kth largest/smallest', ya 'priority' ka zikr ho — Heap ke baare mein sochein."

12. Binary Search

THEORY

Binary Search ek bohat efficient searching algorithm hai, lekin sirf **sorted array** par kaam karta hai. Idea ye hai ke har step mein search space ko **aadha (half)** kar dein, isi liye ye $O(\log n)$ mein result de deta hai — chahe array kitna bhi bara kyun na ho.

Kaam Kaise Karta Hai

Socho aapko dictionary mein ek lafz dhoondna hai. Aap poori dictionary shuru se end tak nahi parhte (ye Linear Search hota, $O(n)$) — balke aap beech se khol kar dekhte hain, agar aapka lafz us se pehle ata hai to left half mein jate hain, warna right half mein. Yehi Binary Search hai.

```
def binary_search(arr, target):
    left, right = 0, len(arr) - 1

    while left <= right:
        mid = (left + right) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            left = mid + 1
        else:
            right = mid - 1

    return -1 # element nahi mila

arr = [1, 3, 5, 7, 9, 11, 13]
print(binary_search(arr, 7)) # Output: 3 (index)
print(binary_search(arr, 4)) # Output: -1
```

Time: $O(\log n)$ | Space: $O(1)$ (iterative version) | Recursive version: $O(\log n)$ space (call stack ki wajah se)

Zaroori Shart: Binary Search sirf **sorted** data par kaam karta hai. Agar array sorted nahi hai, to pehle sort karna paray ga ($O(n \log n)$) — ya phir Linear Search ($O(n)$) use karna behtar hoga.

Kab Use Karein

- Sorted array mein element dhoondna.
- "Find first/last occurrence" type questions.
- **Binary Search on Answer** — ek advanced pattern jahan Binary Search kisi bhi monotonic function par apply kiya jata hai, sirf array search ke liye nahi (jaise: "minimum capacity to ship packages in D days" type problems).

Interview mein yaad rakhein: Jab bhi "sorted array" ka zikr ho aur time complexity $O(\log n)$ chahiye ho, Binary Search ke baare mein turant sochein.

13. Topological Sort

THEORY

Topological Sort ek aisi ordering hai jo sirf **DAG (Directed Acyclic Graph)** par apply hoti hai — matlab graph directed ho aur usme koi cycle na ho. Ye us tarah nodes ko arrange karta hai ke agar edge $A \rightarrow B$ hai, to ordering mein A hamesha B se pehle aaye.

Real-Life Example (Ye Samajhna Sab Se Asaan Hai)

Socho aap university mein courses le rahe hain. Kuch courses ke **prerequisites** hotay hain — jaise "Data Structures" lene se pehle "Intro to Programming" lena zaroori hai. Topological Sort aapko ek aisa order deta hai jisme aap saare courses is tarah le sakein ke koi bhi course apne prerequisite se pehle na aaye.

Do Tareeqay (Approaches)

- **DFS-based:** Har node par DFS karein, aur jab node ka poora processing complete ho jaye (saare children visit ho chukay hon), usko ek stack mein push kar dein. Aakhir mein stack ko pop kar ke reverse order nikal lein — yehi topological order hai.
- **Kahn's Algorithm (BFS-based):** Har node ka **in-degree** (kitne edges us par aa rahe hain) calculate karein. Jin nodes ka in-degree 0 ho (matlab koi prerequisite nahi), unhein Queue mein daal dein. Queue se node nikalein, result mein add karein, aur uske neighbors ka in-degree 1 kam kar dein — agar kisi neighbor ka in-degree 0 ho jaye, usay bhi queue mein daal dein.

```
# Kahn's Algorithm (BFS-based Topological Sort)
from collections import deque

def topological_sort(graph, num_nodes):
    in_degree = {node: 0 for node in graph}
    for node in graph:
        for neighbor in graph[node]:
            in_degree[neighbor] += 1

    queue = deque([node for node in graph if in_degree[node] == 0])
    result = []

    while queue:
        node = queue.popleft()
        result.append(node)
        for neighbor in graph[node]:
            in_degree[neighbor] -= 1
            if in_degree[neighbor] == 0:
                queue.append(neighbor)

    return result if len(result) == num_nodes else [] # empty list = cycle exists

graph = {'A': ['B', 'C'], 'B': ['D'], 'C': ['D'], 'D': []}
print(topological_sort(graph, 4)) # Output: ['A', 'B', 'C', 'D'] (ya milta julta valid order)
```

Time: $O(V + E)$ | Space: $O(V)$

Kab Use Karein

- Course scheduling (prerequisites ke sath).
- Build systems / task scheduling (jaise Makefile mein dependencies).
- Package manager dependency resolution (jaise npm, pip packages install karte waqt).

Bonus fact: Agar Kahn's Algorithm ke end mein result mein saare nodes shamil nahi hotay, iska matlab graph mein **cycle** hai — is liye ye algorithm cycle detection ke liye bhi use hota hai.

14. Basic Dynamic Programming (DP)

THEORY

Dynamic Programming ek technique hai jo bare problems ko chote sub-problems mein tor kar solve karti hai, aur **har sub-problem ka answer store (cache) kar leti hai** taake usay dobara calculate na karna paray. Ye us waqt use hota hai jab problem mein **overlapping subproblems** aur **optimal substructure** ho.

Do Zaroori Concepts

- **Overlapping Subproblems:** Same chota problem baar baar solve karna par raha ho. Jaise Fibonacci mein `fib(5)` calculate karne ke liye `fib(3)` multiple baar calculate hota hai agar hum simple recursion use karein.
- **Optimal Substructure:** Bare problem ka best solution, chote sub-problems ke best solutions se ban sakta ho.

Classic Example: Fibonacci Numbers

Simple Recursion (Bina DP ke) — Bohat Slow:

```
def fib_slow(n):
    if n <= 1:
        return n
    return fib_slow(n - 1) + fib_slow(n - 2)
# Time:  $O(2^n)$  - kyunke same values baar baar recalculate hoti hain
```

DP Approach #1 — Memoization (Top-Down): Recursion hi use karein, lekin results ko ek dictionary/array mein "cache" kar lein.

```
def fib_memo(n, memo={}):
    if n in memo:
        return memo[n]
    if n <= 1:
        return n
    memo[n] = fib_memo(n - 1, memo) + fib_memo(n - 2, memo)
    return memo[n]
# Time:  $O(n)$  - har value sirf ek baar calculate hoti hai
```

DP Approach #2 — Tabulation (Bottom-Up): Recursion ke bajaye loop se, chote se bare values tak table fill karein.

```
def fib_tab(n):
    if n <= 1:
        return n
    dp = [0] * (n + 1)
    dp[1] = 1
    for i in range(2, n + 1):
        dp[i] = dp[i - 1] + dp[i - 2]
    return dp[n]
# Time:  $O(n)$ , Space:  $O(n)$  - aur agar sirf last 2 values rakhein to Space  $O(1)$  bhi ho sakta hai
```

Memoization aur Tabulation dono $O(n)$ time dete hain — Simple recursion ke $O(2^n)$ se bohat behtar

Memoization vs Tabulation — Farq

Memoization Top-Down. Recursion use karta hai. Cache dictionary/array mein results store hotay hain. Likhna asaan hota hai (recursion se sochna natural hai).

Tabulation

Bottom-Up. Loop use karta hai. Table (array) chote se bare values tak fill hota hai. Recursion stack overflow ka risk nahi hota.

Kuch Aam DP Problems (Basic Level)

- **Climbing Stairs** — Ek time mein 1 ya 2 steps chal saktay hain, n steps tak pohanchne ke kitne tareeqay hain? (Fibonacci jaisa hi pattern)
- **House Robber** — Adjacent houses rob nahi kar saktay, maximum kitna amount rob kar saktay hain?
- **Maximum Subarray Sum (Kadane's Algorithm)** — Ye Array section mein already solve kiya, ye bhi ek DP problem hai (1D DP).
- **Coin Change** — Minimum kitne coins chahiye ek target amount banane ke liye.
- **Longest Common Subsequence** — Do strings mein sab se lambi common subsequence.

Interview mein DP identify karne ka tareeqa: Agar question mein "minimum/maximum ways", "count number of ways", ya "optimal solution" jaisi cheezein ho aur brute force mein repeated calculations nazar aa rahi hon — to DP ke baare mein sochein. Pehle simple recursion likhein, phir memoization add kar dein.

15. Bonus: Interview Tips & Cheat Sheet

BONUS

Time Complexity Cheat Sheet

Array Access	$O(1)$
Array Search (unsorted)	$O(n)$
Binary Search (sorted)	$O(\log n)$
HashMap Insert/Search/Delete	$O(1)$ average
Stack/Queue Push/Pop	$O(1)$
BFS/DFS (Graph/Tree)	$O(V + E)$
Heap Insert/Extract	$O(\log n)$
Sorting (average, e.g. Merge/Quick Sort)	$O(n \log n)$
Basic DP (1D)	$O(n)$ time, $O(n)$ or $O(1)$ space

AI Engineer ke Liye Extra Tip

Agar aap AI Engineer role ke liye interview de rahe hain, to DSA ke sath sath ye bhi expect karein:

- Python fundamentals (list comprehension, generators, numpy arrays vectorization).
- Time/space complexity ka analysis — har solution ke baad khud puchhein "iski complexity kya hai?"
- Kabhi kabhi DSA questions ko ML context mein wrap kiya jata hai (jaise: "given embeddings, find k-nearest" — ye Heap/Sorting ka application hai).

Interview Se Pehle Ka Checklist

- Har topic ka **ek-line summary** yaad rakhein (jaise is guide mein "tip" boxes diye hain).
- Array aur String ke saare practical questions khud se code karein, bina dekhe.
- Sochte waqt **zor se bolein** (think aloud) — interviewer process dekhna chahta hai, sirf final answer nahi.
- Edge cases hamesha mention karein: empty input, single element, duplicates, negative numbers.
- Agar stuck ho jayein, brute force approach bata dein pehle — kuch na batane se better hai kuch bata dena, phir optimize karne ki koshish karein.

All the best! Confidence sab se zaroori cheez hai. Aap ne ye guide poori parh li, ab practice consistency se karein — roz thora thora karein, last minute mein sab kuch cram na karein. Aap ye kar saktay hain!